

```

from math import *
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib.pyplot as plt
import numpy as np

class Polynome:
    """ définition d'un polynôme avec des coefficients réels"""
    #Polynome([[deg, coeff]])

    def __init__(self, L=[]):
        """ Définit la façon dont se structure un élément de la classe polynôme. L est une liste de couples [degré,
coefficient]"""
        self.termes = L

    def __repr__(self):
        """permet d'afficher tous les résultats de la même façon, sous forme d'une chaîne de caractères"""
        return self.__str__()

    def __str__(self):
        """ Définit la façon dont s'affiche un polynôme avec la commande print """
        affichage = []
        (self.termes).sort() #ordonne les termes
        for k in range(len(self.termes)):

            if isinstance(self.termes[k][1], float) and self.termes[k][1].is_integer(): #permet de bien afficher
pour la division int et non float
                self.termes[k][1] = int(self.termes[k][1])

            if self.termes[k][1] == 0 : #enlève termes avec coeff 0
continue #évite d'avoir un espace libre
            if self.termes[k][1] != 0 and self.termes[k][1] != 1 and self.termes[k][1] != -1: #pour les autres termes
sauf ceux de degré 1 et -1
                if self.termes[k][0] == 0:
                    affichage.append(f"{str(self.termes[k][1])}") #enlève le x^0
                elif self.termes[k][0] == 1:
                    affichage.append(f"{str(self.termes[k][1])}*X") #remplace x^1 par x
                else :
                    affichage.append(f"{str(self.termes[k][1])}*X^{str(self.termes[k][0])}")
            if self.termes[k][1] != 0 and self.termes[k][1] == 1: #pour le coefficient 1, évite le x*1 inutile
                if self.termes[k][0] == 0:
                    affichage.append(f"{str(self.termes[k][1])}") #enlève le x^0
                elif self.termes[k][0] == 1:
                    affichage.append(f"X") #remplace x^1 par x
                else :
                    affichage.append(f"X^{self.termes[k][0]}")
            if self.termes[k][1] != 0 and self.termes[k][1] == -1: #pour le coefficient -1, évite le x*(-1) inutile
                if self.termes[k][0] == 0:
                    affichage.append(f"{str(self.termes[k][1])}") #enlève le x^0
                elif self.termes[k][0] == 1:
                    affichage.append(f"-X") #remplace x^1 par x
                else :
                    affichage.append(f"-X^{self.termes[k][0]}")

        return (" + ".join(affichage).replace("+ -","- ")) #met des + entre chaque terme et remplace les + - par des -

# Evaluation en un point

def point(self, abscisse):
    """ Définit l'évaluation en un point """
    ordonnee = 0
    for k in range(len(self.termes)):
        ordonnee += self.termes[k][1]*(abscisse**(self.termes[k][0]))
    return(ordonnee)

def horner(self, abscisse):
    """définit l'évaluation d'un polynôme en un point à l'aide de la méthode d'Horner """
    n=0
    for i in range(len(self.termes)): #détermine le degré de self
        if self.termes[i][0]>=n:
            n=self.termes[i][0]
    for j in range(len(self.termes)):
        if self.termes[j][0]==n:
            p=j #emplacement du degré max n
    m=self.termes[p][1]
    d=n-1
    for i in range(n):
        m=m*abscisse
        k=-1 #pour cas où il n'existe pas, le coefficient du degré est égal à 0
        for j in range(len(self.termes)): #on veut déterminer où est l'emplacement du coefficient de degré d
            if self.termes[j][0]==d:
                k=j
        if k !=-1:
            m=m+self.termes[k][1]
        if k== -1:

```

```

        m=m
        d=d-1
        return(m)

# Addition et soustraction

def __add__(self, autre):
    """ Définit l'addition de deux polynômes à l'aide du signe + """
    #INITIALISATION
    A = []
    B = self.termes.copy()
    C = autre.termes.copy()

    #DÉBUT DE L'ADDITION
    for k in range(len(autre.termes)):
        for i in range(len(self.termes)):
            #la boucle permet de parcourir tous les termes des polynômes
            if self.termes[i][0] == autre.termes[k][0]:
                H = self.termes[i][1] + autre.termes[k][1]
                A.append([self.termes[i][0], H]) #la liste A possède les termes issus de l'addition
                p=B.index(self.termes[i])      #le p et le q servent à avoir l'indice de terme qu'on veut dans la
liste
                B.pop(p)
                q=C.index(autre.termes[k])
                C.pop(q)
            if (A+B+C)[k][1]==0:                #le if retourne la valeur 0 si la somme est égale à 0
                return('0')
    return(Polynome(A+B+C))

def __sub__(self, autre):
    """ Définit la soustraction de deux polynômes à l'aide du signe - """
    #INITIALISATION
    A = []
    B = self.termes.copy()
    C = autre.termes.copy()
    C_neg = []                                #pour les termes de C restants

    #DÉBUT DE LA SOUSTRACTION
    for k in range(len(autre.termes)):
        #les deux boucles parcourent les termes des polynômes et ainsi les
degrés de chaque polynôme sont comparés
        for i in range(len(self.termes)):
            if self.termes[i][0] == autre.termes[k][0]:
                H = self.termes[i][1] - autre.termes[k][1]
                A.append([self.termes[i][0], H])
                p=B.index(self.termes[i])
                B.pop(p)
                q=C.index(autre.termes[k])
                C.pop(q)

    #GESTION DES TERMES RESTANTS DU SECOND POLYNÔME
    for t in range(len(C)):
        #met les termes de C qui restent en valeur négative car ils sont soustraits
        C_neg.append([C[t][0], -C[t][1]])
    D = []                                     #la liste D ne contient que des coefficients non nuls
    for t in range(len(A+B+C_neg)):
        if (A+B+C_neg)[t][1] != 0:
            D.append((A+B+C_neg)[t])
    if D == []:                               #si D est vide, le polynôme est nul
        return(Polynome([]))                 #retourne le polynôme nul
    return(Polynome(A+B+ C_neg))

# multiplication et division

def __mul__(self, autre):
    """ Définit la multiplication de deux polynômes à l'aide du signe * """
    #INITIALISATION
    A = [] #initialisation de la liste correspondant au produit des polynômes
    n=0    #initialisation du degré du polynôme self
    m=0    #initialisation du degré du polynôme autre
    for i in range (len(self.termes)): #détermine le degré du polynôme self
        if self.termes[i][0]>=n:
            n=self.termes[i][0]
    for i in range (len(autre.termes)): #détermine le degré du polynôme autre
        if autre.termes[i][0]>=m:
            m=autre.termes[i][0]

    #MULTIPLICATION
    for k in range(n+m+1):
        for i in range(k+1):
            p=-1 #permet de repérer les coefficients nuls dans le polynôme self
            q=-1 #permet de repérer les coefficients nuls dans le polynôme autre
            for j in range(len(self.termes)): #détermine l'indice de la sous-liste associé au degré i du polynôme
self
                if self.termes[j][0]==i:
                    p=j #p est l'indice du terme de degré i
            for j in range(len(autre.termes)): #détermine l'indice de la sous-liste associée au degré i du polynôme
autre
                if autre.termes[j][0]==k-i:
                    q=j #q est l'indice du terme de degré k-i

```

```

        if p==-1 or q==-1 :
            c=0 #on associe le coefficient 0 pour les degrés qui n'ont pas de sous-liste correspondantes dans
les polynômes self et autre
        else:
            c=self.termes[p][1]*autre.termes[q][1] #calcul des coefficients du produit des polynômes
            A.append([k,c]) #on complète la liste au fur et à mesure que l'on calcule le coefficient associé à
chaque degré

#ÉCRITURE DU RÉSULTAT PLUS PROPREMENT
B = [] #initialisation d'une nouvelle liste correspondant au produit des polynômes, qui additionne les termes de même
degré
while A!=[]:
    d = A[0][0]
    s = 0 #initialisation de la somme des coefficients associés au même degré
    i = 0
    while i < len(A):
        if A[i][0] == d:
            s += A[i][1] #on fait la somme de tous les coefficients associés au degré d
            A.pop(i) #on supprime l'élément de la liste A
        else:
            i += 1
    B.append([d, s]) #on réécrit la liste avec les coefficients additionnés
return Polynome(B)

def x(self, reel):
    """ Définit la multiplication d'un polynôme par un réel """
    A = self.termes.copy()
    for k in range(len(self.termes)):
        A[k][1] = A[k][1]*reel #on multiplie chaque coefficient par le réel considéré
    return(Polynome(A))

def __truediv__(self, diviseur):
    """définit la division d'un polynôme par un autre à l'aide du signe / """
    #INITIALISATION
    P = self.termes.copy()
    div_termes = [t for t in diviseur.termes if t[1] != 0] #évite la division par 0 et ne modifie pas le diviseur
original
    n=0
    m=0
    Q = [] #pour le quotient
    R = [] #pour le reste
    for i in range (len(self.termes)): #détermine le degré de self
        if self.termes[i][0]>=n:
            n=self.termes[i][0]
    for g in range (len(div_termes)): #détermine le degré du diviseur
        if div_termes[g][0]>=m:
            m=div_termes[g][0]
    if n < m:
        return('le degré du polynôme doit être supérieur ou égal à celui du diviseur')
    while n >= m:
        #ÉVITE LA DIVISION PAR 0
        for i in range (len(div_termes)-1,-1,-1):
            if div_termes[i][1] == 0:
                div_termes.pop(i) #enlève les termes qui ont un coefficient 0

        #DÉBUT DIVISION
        DQ = n - m #degré du quotient
        p = -1
        q = -1
        for j in range(len(P)):
            if P[j][0]==n:
                p=j #emplacement du degré max n

        for k in range(len(div_termes)): #étapes
            if div_termes[k][0]==m:
                q=k #emplacement du degré max m
                CQ = (P[p][1])/(div_termes[q][1])
                Q.append([DQ,CQ]) #termes du quotient
                produit = Polynome(div_termes) * Polynome([DQ,CQ]) #calcul D[1]*Q
                R = Polynome(P) - produit #reste
                P = [term for term in R.termes if term[1] != 0] #le polynôme à diviser devient le
reste
            if P == [] :
                break #si jamais il n'y a plus rien à diviser

        for i in range (len(P)-1,-1,-1):
            if P[i][1] == 0:
                P.pop(i) #enlève les termes qui ont un coefficient 0

        if P == []:
            break #si il n'y a plus rien à diviser
        if P:
            n=0
            for i in range (len(P)):
                #détermine le degré de self

```

```

        if P[i][0]>=n:
            n=P[i][0]

    return(Polynome(Q), R)

# Racines

def racine(self):
    """ définit la détermination des racines pour un polynôme d'ordre 1 ou 2 """
    #INITIALISATION
    n=0
    (self.termes).sort() #ordonne les termes
    for i in range (len(self.termes)): #détermine le degré, l'ordre du polynôme
        if self.termes[i][0]>=n:
            n=self.termes[i][0]

    #POLYNÔME D'ORDRE 2
    if n == 2: #ordre 2
        p = -1
        q = -1
        for j in range(len(self.termes)):
            if self.termes[j][0]==0:
                p=j #détermine l'emplacement du degré 0
        for j in range(len(self.termes)):
            if self.termes[j][0]==1:
                q=j #emplacement degré 1
        for j in range(len(self.termes)):
            if self.termes[j][0]==2:
                r=j #emplacement degré 2
        if p == -1 and q != -1: #si il n'y a pas de coefficient associé au degré 0
            deltap = (self.termes[q][1]**2)
            racine1 = (-1*self.termes[q][1] + sqrt(deltap))/(2*self.termes[r][1])
            racine2 = (-1*self.termes[q][1] - sqrt(deltap))/(2*self.termes[r][1])

        elif q == -1 and p != -1: #si il n'y a pas de coefficient associé au degré 1
            deltaq = (-4)*(self.termes[r][1])*(self.termes[p][1])
            racine1 = (sqrt(deltaq))/(2*self.termes[r][1])
            racine2 = (-sqrt(deltaq))/(2*self.termes[r][1])

        elif q == -1 and p == -1: #si il n'y a pas de coefficient associé au degré 2
            racine1 = 0
            racine2 = 0

        else: #s'il y a tous les coefficients
            delta = (self.termes[q][1]**2) - 4*(self.termes[r][1])*(self.termes[p][1])
            if delta>0:
                racine1 = (-1*self.termes[q][1] + sqrt(delta))/(2*self.termes[r][1])
                racine2 = (-1*self.termes[q][1] - sqrt(delta))/(2*self.termes[r][1])
            if delta ==0:
                racine1 = (-1*self.termes[q][1])/(2*self.termes[r][1])
                racine2 = racine1
            if delta < 0:
                racine1 = (-1*self.termes[q][1] + 1j*sqrt(-delta))/(2*self.termes[r][1]) #représente le i complexe en
python                racine2 = (-1*self.termes[q][1] - 1j*sqrt(-delta))/(2*self.termes[r][1])
            return(racine1, racine2)

    #POLYNÔME D'ORDRE SUPÉRIEUR À 2 OU D'ORDRE 0
    if n > 2 or n==0: #ordre supérieur à 2 ou ordre 0
        return('il faut un polynôme d ordre 2 ou 1')

    #POLYNÔME D'ORDRE 1
    if n == 1: #ordre 1
        p = -1
        q = -1
        for j in range(len(self.termes)):
            if self.termes[j][0]==0:
                p=j #emplacement du degré 0
        for j in range(len(self.termes)):
            if self.termes[j][0]==1:
                q=j #emplacement du degré 1
        if q == -1 and p !=-1: #s'il n'y a pas de coefficient associé au degré 1
            return('il faut un polynôme d ordre 2 ou 1')
        if p == -1: #s'il n'y a pas de coefficient associé au degré 0
            racine = 0
        else :
            racine = (-self.termes[p][1])/(self.termes[q][1])
        return(racine)

```

```

def racinesev(self):
    """Détermine les racines évidentes d'un polynôme et factorise le polynôme """
    L = []
    L1=[]

    #CALCUL DES RACINES ÉVIDENTES ENTRE 4 ET -4
    for i in range (-4,5):
        P = 0
        for k in range(len(self.termes)):
            P += (self.termes[k][1] * (i**(self.termes[k][0]))) #calcule le polynôme pour x = i
        if P == 0:
            L.append(i) #si le polynôme s'annule en i, on ajoute cette racine évidente à la liste L

    #POLYNÔMES DE FORME X - RACINE ÉVIDENTE
    for j in range(len(L)):
        L1+=([[1,1],[0,-L[j]]])

    #DÉBUT FACTORISATION --> DIVISION DU POLYNÔME SELF PAR LES POLYNÔMES LIÉS AUX RACINES ÉVIDENTES
    P = self.termes.copy()
    for l in range(len(L1)):
        D = (L1[l]).copy()
        n=0 #initialisation du degré du polynôme self
        m=1 #degré du polynôme associé à la racine évidente
        Q = [] #initialisation de la liste pour le quotient
        R = [] #initialisation de la liste pour le reste
        for i in range (len(P)):#détermine le degré du polynôme self
            if P[i][0]>=n:
                n=P[i][0]

        while n >= m: #tant que le degré du reste est supérieur au degré du diviseur
            DQ = n - m #degré du quotient
            p = -1
            q = -1
            for j in range(len(P)):
                if P[j][0]==n:
                    p=j #p = emplacement du degré max n

            for k in range(2):
                if L1[l][k][0]==m:
                    q=k #q = emplacement du degré max m
                    CQ = (P[p][1])/(L1[l][q][1]) #division du coefficient associé au degré le plus élevé du dividende par
le coefficient le plus élevé du diviseur
                    Q.append([DQ,CQ]) #on construit la liste associée au quotient
                    produit = Polynome(D) * Polynome([DQ,CQ]) #on fait le produit du diviseur par le terme du quotient
qui vient d'être déterminé
                    R = Polynome(P) - produit #calcul du reste
                    P = R.termes.copy() #le reste devient le polynôme à diviser
                    if P == [] : #si jamais il n'y a plus rien à diviser, polynôme nul
                        break

            for i in range (len(P)-1,-1,-1):
                if P[i][1] == 0:
                    P.pop(i) #enlève les termes qui ont un coefficient égal à 0

            if P == []: #si jamais il n'y a plus rien à diviser
                break

        n =0
        for i in range(len(P)): #détermine le degré de P puisque il est remis à jour à chaque étape
            if P[i][0]>=n:
                n=P[i][0]
        P=Q #on recommence avec tous les polynômes associés aux racines évidentes pour avoir le dernier polynôme (celui
sans les racines évidentes)

        if len(L1) == 0:
            P = self.termes.copy() #si le polynôme n'a pas de racines, P redevient le polynôme self

    #AFFICHAGE
    facteurs = "" #permet un bon affichage de la factorisation
    for m in range(len(L1)):
        facteurs += f"({Polynome(L1[m])})"
    if P != [[0,1]]: #évite d'afficher le facteur constant 1 (si le polynôme self peut s'écrire sous la forme de
produits de produits associés à des racines évidentes)
        facteurs += f"({Polynome(P)})"

    B = ""
    if len(L)>1:
        B = 'Les racines évidentes sont ' + ', '.join(str(x) for x in L) + ' .'
    if len(L) == 1:
        B = 'La racine évidente est ' + ', '.join(str(x) for x in L) + ' .'
    if len(L) == 0 :
        B = f'Pas de racines évidentes.'

    return(B, facteurs)

```

```

def Sturm(self,a,b):
    """ définit l'utilisation du théorème de Sturm """
    P0 = self #le polynôme dont on cherche le nombre de racines

    #CRÉE UNE LISTE AVEC LES POLYNÔMES DE LA SUITE DE STURM
    P1 = P0.derive() #P1
    L = [P0, P1] #suite de Sturm
    while P1.termes != 0:
        Q, R = P0/P1
        if R.termes == []: #si le reste est égal à 0 on s'arrête
            break
        P2 = R.x*(-1) #multiplie le reste par -1
        L.append(P2) #rajoute le polynôme P2 à la liste de Sturm
        P0 = P1
        P1 = P2

    #CALCULE EN a ET EN b, ÉVALUE LES SIGNES
    A=[] #liste qui contient les résultats de l'évaluation des polynômes en a
    B=[]
    for i in range(len(L)): #parcourt tous les polynômes de la suite de Sturm
        u = L[i].horner(a) #évalue chaque polynôme en a grâce à la méthode d'Horner
        v = L[i].horner(b)
        if u > 0:
            A.append(1) #ajoute 1 à la liste A si le nombre obtenu est positif
        elif u < 0:
            A.append(-1) #ajoute -1 à la liste A si le nombre obtenu est négatif
        if v > 0:
            B.append(1)
        elif v < 0:
            B.append(-1)

    #CALCULE LES CHANGEMENTS DE SIGNES EN A et B
    c = 0 #compte le nombre de changements de signes en a
    for i in range(len(A)-1):
        if A[i] != A[i+1]:
            c += 1 #dans la liste A, si un terme a un signe différent du terme suivant on ajoute 1 à c

    d = 0 #compte le nombre de changements de signes en b
    for i in range(len(B)-1):
        if B[i] != B[i+1]:
            d += 1 #dans la liste B, si un terme a un signe différent du terme suivant on ajoute 1 à d

    #CALCUL FINAL
    nb = c - d #nombre de changements de signes en a - nombre de changements de signes en b

    return(f'Ce polynôme a {nb} racine(s) entre {a} et {b}.')

```

```

def bairstow(self, eps=1e-7, imax=50):
    """ Détermine les racines d'un polynôme avec la méthode de Bairstow """

    #eps = précision pour arrêter les calculs
    #imax = nombre max d'itérations pour éviter boucle infinie
    P = Polynome([t for t in self.termes if abs(t[1]) > 1e-10]) #On crée un polynôme P en supprimant les coefficients

    racines = [] #initialisation de la liste finale des racines trouvées

    while True:
        #degré du polynôme
        if P.termes != []:
            deg = max([t[0] for t in P.termes])
        else:
            deg = 0

        #cas simples
        if deg <= 2:
            res = P.racine() #détermine les racines du polynôme du second degré ou de degré inférieur
            if isinstance(res, tuple):
                racines.extend(res) #ajoute les différentes racines à la liste
            else:
                racines.append(res) #ajoute la racine à la liste
            break #permet de sortir de la boucle while

        #INITIALISATION
        p = 0.0
        q = 1.0

        #boucle d'itération pour améliorer p et q
        for i in range(imax):
            #récupération des coefficients
            deg = max([t[0] for t in P.termes]) #recalcul du degré (au cas où P change)
            #tableau des coefficients du polynôme
            a = [0]*(deg+1) #création d'une liste de longueur degré+1 avec que des 0
            #on transforme la liste de [degré, coeff] en tableau indexé
            for d, c in P.termes:

```

~0

```

croissant      a[d] = c      #a devient une liste avec tous les coefficients associés à chaque degré dans l'ordre

# tableaux intermédiaires b et c utilisés par Bairstow
b = [0]*(deg+1)
c = [0]*(deg+1)

#initialisation récursive de b (division synthétique)
b[deg] = a[deg]
b[deg-1] = a[deg-1] + p*b[deg]

for i in range(deg-2, -1, -1):
    b[i] = a[i] + p*b[i+1] + q*b[i+2]

#initialisation récursive de c (dérivées du système)
c[deg] = b[deg]
c[deg-1] = b[deg-1] + p*c[deg]

for i in range(deg-2, -1, -1):
    c[i] = b[i] + p*c[i+1] + q*c[i+2]

#déterminant du système linéaire pour résoudre dp et dq
if deg >= 3:
    det = c[2]*c[2] - c[3]*c[1]
else:
    det = 0

if abs(det) < eps:
    break #système instable → on arrête l'itération

#corrections de p et q (résolution du système Cramer)
dp = (-b[1]*c[2] + b[0]*c[3]) / det
dq = (-b[0]*c[2] + b[1]*c[1]) / det

#mise à jour des paramètres
p += dp
q += dq

#si les corrections sont petites, on ne change presque plus les valeurs → convergence atteinte
if abs(dp) < eps and abs(dq) < eps:
    break

#CONSTRUCTION DU FACTEUR QUADRATIQUE
#on construit le polynôme  $x^2 - p x - q$ 
facteur = Polynome([[2,1], [1,-p], [0,-q]])

#on calcule ses racines
r1, r2 = facteur.racine()
racines.append(r1)
racines.append(r2)

#on divise le polynôme par ce facteur quadratique
Q, R = P / facteur
#on remplace P par le quotient pour recommencer
P = Q

#AFFICHAGE
L = racines
B = ""
if len(L)>1:
    B = 'Les racines sont ' ':'
if len(L) == 1:
    B = 'La racine est ' ':'
if len(L) == 0 :
    B = f'Pas de racines.'

facteurs=racines #permet simplement de pouvoir faire appel à la fonction "afficher_latex_racinesEV" originellement
créée pour la procédure "racinesev"

return(B,facteurs)

# Dérivation et intégration
def derive(self):
    """définit la dérivée simple"""
    A = []
    P = self.termes.copy()
    for i in range(len(self.termes)):
        if P[i][0] == -1:
            continue
        a = P[i][0] - 1      #enlève 1 à chaque degré
        b = P[i][1] * P[i][0] #multiplie chaque coefficient par son degré associé initial
        A.append([a,b])

```

```

        return(Polynome(A))

def derivesucc(self,n):
    """définit la dérivée successive"""
    A = []
    P = self.termes.copy()
    for i in range(len(self.termes)):
        a = P[i][0] - n #enlève n à chaque degré
        b=P[i][1]
        if a <= -1:
            continue #si a est négatif, le terme est ignoré car c'est une constante dérivée donc il est égal à 0
        for k in range(n): #multiplie le coefficient par son degré n fois en enlevant 1 à chaque fois
            b = b * (P[i][0]-k)
        A.append([a,b])

    return(Polynome(A))

def primitive(self):
    """définit la primitive"""
    A = []
    P = self.termes.copy()
    for i in range(len(self.termes)):
        a = P[i][0] + 1 #ajoute 1 à chaque degré
        b = P[i][1]*(1/(P[i][0] +1)) #divise chaque coefficient par son degré associé final
        A.append([a,b])

    return(Polynome(A))

def integrale(self,c,d):
    """définit l'intégration"""
    A = []
    P = self.termes.copy()
    for i in range(len(self.termes)): #calcule la primitive
        a = P[i][0] + 1
        b = P[i][1]*(1/(P[i][0] +1))
        A.append([a,b])
    res=Polynome(A).horner(d)-Polynome(A).horner(c) #fait la différence de la primitive aux deux bornes en utilisant
    l'évaluation en un point

    return(res)

# Arithmétique

def pgcd(self, autre):
    """Détermine le PGCD de deux polynômes """
    A = self
    B = autre

    while B.termes != []: #on effectue les divisions successives jusqu'à ce que le reste soit égal à 0
        Q, R = A / B
        #évite boucle infinie
        if sorted(R.termes) == sorted(B.termes):
            break

        A = B #le diviseur devient le dividende
        B = R #le reste devient le diviseur
        A = Polynome([t for t in A.termes if t[1] != 0])

        #normalisation, permet d'avoir un polynôme unitaire
        m=max([t[0] for t in A.termes]) #détermine le degré du polynôme A
        for j in range(len(self.termes)): #détermine l'indice de la sous-liste associé au degré le plus élevé du
polynôme self
            if self.termes[j][0]==m:
                p=j
        k=A.termes[p][1] #k correspond au coefficient associé au degré le plus élevé

        A2 = A.x(1/k) #on divise le polynôme par le coefficient associé au degré le plus élevé

    return(A2)

def ppm(self, Q):
    """ Détermine le PPCM de deux polynômes """
    G = self.pgcd(Q) # Calcul du PGCD de self et Q
    P = self * Q # Produit des deux polynômes self et Q.
    Q, R = P / G # Division
    return (Q)

```



```

def bezout(self, autre):
    """ Détermine les polynômes U et V impliqués dans l'identité de Bézout """
    #INITIALISATION
    A = self
    B = autre
    Q1 = [] #initialisation de la liste des quotients

    while B.termes != []:
        Q, R = A / B

        #évite boucle infinie
        if sorted(R.termes) == sorted(B.termes):
            break
        Q1.append(Q) #on stocke les quotients successifs
        A = B
        B = R
        A = Polynome([t for t in A.termes if abs(t[1]) > 1e-10])

    #INITIALISATION DE BEZOUT
    rA = [Polynome([[0, 1]]), Polynome([[0, 0]])] # u0 = 1, u1 = 0
    rB = [Polynome([[0, 0]]), Polynome([[0, 1]])] # v0 = 0, v1 = 1

    #RECONSTRUCTION DES RESTES EN FONCTION DE A et B AVEC LES QUOTIENTS
    for i in range(len(Q1)):
        Q = Q1[i]
        u = rA[-2] - (Q * rA[-1])
        v = rB[-2] - (Q * rB[-1])
        rA.append(u)
        rB.append(v)

    #RÉCUPÈRE LES AVANT-DERNIERS ÉLÉMENTS DE LA LISTE
    u_final = rA[-2]
    v_final = rB[-2]

    return (u_final, v_final)

```

#INTERFACE

```

from tkinter import*
from tkinter import ttk
import tkinter as tk
import matplotlib as mpl
import matplotlib.pyplot as plt
from sympy import *

#création de la fenêtre principale
fenetre = Tk() #création d'une fenêtre graphique principale en utilisant la bibliothèque tkinter
fenetre.title('Polynome')
fenetre.geometry("1500x650")
fenetre.configure(bg = 'beige')

#espace pour rentrer les polynômes
polynome1=StringVar()
P1 = ttk.Entry(textvariable=polynome1)
polynome2=StringVar()
P2 = ttk.Entry(textvariable=polynome2)

#case résultat
resultat = StringVar()
label_resultat = Label(fenetre, textvariable=resultat,width=50)

#définition des programmes
def addition():
    try:
        Val1=P1.get() #récupère les valeurs rentrées
        Val2=P2.get()
        res=Polynome(eval(Val1))+Polynome(eval(Val2)) #la commande eval permet de convertir la valeur rentrée en liste
        resultat.set(str(res))
    
```

```

        afficher_latex(res)      #affiche le résultat sous forme LaTeX
        afficher_graphique(res) #affiche le graphique
    except:
        afficher_latex("Erreur de saisie") #si les polynômes ne sont pas bien saisis

def soustraction():
    try:
        Val1=P1.get()
        Val2=P2.get()
        res=Polynome(eval(Val1))-Polynome(eval(Val2))
        resultat.set(str(res))
        afficher_latex(res)
        afficher_graphique(res)
    except:
        afficher_latex("Erreur de saisie")

def hornerPoint():
    fenetre1=Tk() #création d'une nouvelle fenêtre pour rentrer la valeur de x
    fenetre1.configure(bg = 'turquoise')
    fenetre1.geometry("300x150")
    point=StringVar()
    P=ttk.Entry(fenetre1,textvariable=point)
    P.place(x=80,y=50)
    tk.Label(fenetre1,text="Valeur de x",fg='black', bg='turquoise', font=('arial',12)).place(x=100,y=10)
    def ok(): #pour faire le calcul
        try:
            Val1=P1.get()
            Val2=P.get()
            res=(Polynome(eval(Val1))).horner(eval(Val2))
            resultat.set(str(res))
            afficher_latex(res)
            pas_de_graphique()
        except:
            afficher_latex("Erreur de saisie")
    fenetre1.destroy() #efface la fenêtre créée
    boutonok=Button(fenetre1, text="Calculer", width=20,command=ok)
    boutonok.place(x=70,y=90)

def multiplication():
    try:
        Val1=P1.get()
        Val2=P2.get()
        res=Polynome(eval(Val1))*Polynome(eval(Val2))
        resultat.set(str(res))
        afficher_latex(res)
        afficher_graphique(res)
    except:
        afficher_latex("Erreur de saisie")

def multiReel():
    fenetre1=Tk() #création d'une nouvelle fenêtre pour rentrer la valeur du facteur
    fenetre1.configure(bg = 'turquoise')
    fenetre1.geometry("300x150")
    point=StringVar()
    P=ttk.Entry(fenetre1,textvariable=point)
    P.place(x=80,y=50)
    tk.Label(fenetre1,text="Valeur du facteur",fg='black', bg='turquoise', font=('arial',12)).place(x=80,y=10)
    def ok():
        try:
            Val1=P1.get()
            Val2=P.get()
            res=(Polynome(eval(Val1))).x(eval(Val2))
            resultat.set(str(res))
            afficher_latex(res)
            afficher_graphique(res)
        except:
            afficher_latex("Erreur de saisie")
    fenetre1.destroy()
    boutonok=Button(fenetre1, text="Calculer", width=20,command=ok)
    boutonok.place(x=70,y=90)

def racine2():
    try:
        Val1=P1.get()
        res=Polynome(eval(Val1)).racine()
        resultat.set(str(res))
        afficher_latex(res)
        pas_de_graphique()
    except:
        afficher_latex("Erreur de saisie")

def evRacine():
    try:
        Val1=P1.get()
        B, facteurs = Polynome(eval(Val1)).racinesev()
        afficher_latex_racinesEV(B, facteurs)
        pas_de_graphique()

```

```

except:
    afficher_latex("Erreur de saisie")

def division():
    try:
        Val1=P1.get()
        Val2=P2.get()
        Q,R = Polynome(eval(Val1))/Polynome(eval(Val2))
        resultat.set(f"Quotient : {Q} | Reste : {R}")
        afficher_latex_division(Q,R)
        pas_de_graphique()
    except:
        afficher_latex("Erreur de saisie")

def primitiver():
    try:
        Val1=P1.get()
        res=Polynome(eval(Val1)).primitive()
        resultat.set(str(res))
        afficher_latex(res)
        afficher_graphique(res)
    except:
        afficher_latex("Erreur de saisie")

def derivation():
    try:
        Val1=P1.get()
        res=Polynome(eval(Val1)).derive()
        resultat.set(str(res))
        afficher_latex(res)
        afficher_graphique(res)
    except:
        afficher_latex("Erreur de saisie")

def Sturm():
    fenetre1=Tk() #création d'une nouvelle fenêtre pour rentrer la valeur de l'intervalle
    fenetre1.configure(bg = 'turquoise')
    fenetre1.geometry("300x150")
    point=StringVar()
    P=ttk.Entry(fenetre1,textvariable=point)
    P.place(x=80,y=50)
    tk.Label(fenetre1,text="Intervalle sous la forme (... , ...)",fg='black', bg='turquoise',
font=('arial',12)).place(x=60,y=10)
    def ok():
        try:
            Val1=P1.get()
            Val2=P.get()
            a, b = eval(Val2)
            res=Polynome(eval(Val1)).Sturm(a,b)
            resultat.set(str(res))
            afficher_latex(res)
            pas_de_graphique()

        except:
            afficher_latex("Erreur de saisie")
    fenetre1.destroy()
    boutonok=Button(fenetre1, text="Chercher", width=20,command=ok)
    boutonok.place(x=70,y=90)

def pgcdd():
    try:
        Val1 = P1.get()
        Val2 = P2.get()
        res = (Polynome(eval(Val1))).pgcd(Polynome(eval(Val2)))
        resultat.set(str(res))
        afficher_latex(res)
        afficher_graphique(res)
    except:
        afficher_latex("Erreur de saisie")

def DerSucc():
    fenetre1=Tk() #création d'une nouvelle fenêtre pour rentrer la valeur de l'ordre
    fenetre1.configure(bg = 'turquoise')
    fenetre1.geometry("300x150")
    point=StringVar()
    P=ttk.Entry(fenetre1,textvariable=point)
    P.place(x=80,y=50)
    tk.Label(fenetre1,text="Ordre de la dérivée",fg='black', bg='turquoise', font=('arial',12)).place(x=75,y=10)
    def ok():
        try:
            Val1=P1.get()
            Val2=P.get()
            res=Polynome(eval(Val1)).derivesucc(eval(Val2))
            resultat.set(str(res))
            afficher_latex(res)

```

```

        afficher_graphique(res)
    except:
        afficher_latex("Erreur de saisie")

    fenetre1.destroy()
boutonok=Button(fenetre1, text="Calculer", width=20,command=ok)
boutonok.place(x=70,y=90)

def bezoutt():
    try:
        Val1=P1.get()
        Val2=P2.get()
        Q, R =(Polynome(eval(Val1))).bezout(Polynome(eval(Val2)))
        resultat.set(f"Quotient : {Q} | Reste : {R}")
        afficher_latex_division(Q,R)
        pas_de_graphique()
    except:
        afficher_latex("Erreur de saisie")

def ppcm():
    try:
        Val1 = P1.get()
        Val2 = P2.get()
        res = (Polynome(eval(Val1))).ppcm(Polynome(eval(Val2)))
        resultat.set(str(res))
        afficher_latex(res)
        afficher_graphique(res)
    except:
        afficher_latex("Erreur de saisie")

def bairstow():
    fenetre2=Tk() #création d'une nouvelle fenêtre pour choisir d'entrer des valeurs ou non
    fenetre2.configure(bg = 'turquoise')
    fenetre2.geometry("300x150")
    def initial():
        try:
            Val1=P1.get()
            B, racines = (Polynome(eval(Val1))).bairstow()
            resultat.set(B + " " + " ", ".join(str(r) for r in racines))
            afficher_latex_racinesEV(B, racines)
            pas_de_graphique()
        except:
            afficher_latex("Erreur de saisie")
    fenetre2.destroy()
def saisir():
    fenetre1=Tk() #création d'une nouvelle fenêtre pour rentrer la valeur de précision et Imax
    fenetre1.configure(bg = 'turquoise')
    fenetre1.geometry("435x150")
    point=StringVar()
    P=ttk.Entry(fenetre1,textvariable=point)
    P.place(x=25,y=50)
    tk.Label(fenetre1,text="Précision (Epsilon)",fg='black', bg='turquoise', font=('arial',12)).place(x=20,y=10)
    pointimax=StringVar()
    I=ttk.Entry(fenetre1,textvariable=pointimax)
    I.place(x=240,y=50)
    tk.Label(fenetre1,text="Nombre max d'itération (Imax)",fg='black', bg='turquoise',
font=('arial',12)).place(x=200,y=10)
    def ok():
        try:
            Val1=P1.get()
            Val2=P.get()
            Val3=I.get()
            B, racines = (Polynome(eval(Val1))).bairstow(eval(Val2), eval(Val3))
            resultat.set(B + " " + " ", ".join(str(r) for r in racines))
            afficher_latex_racinesEV(B, racines)
            pas_de_graphique()
        except:
            afficher_latex("Erreur de saisie")
    fenetre1.destroy()
    boutonok=Button(fenetre1, text="Calculer", width=20,command=ok)
    boutonok.place(x=145,y=90)
    fenetre2.destroy()
    boutonsai=Button(fenetre2, text="Saisir des valeurs",command=saisir)
    boutonint=Button(fenetre2, text="Valeurs par défaut",command=initial)
    boutonsai.place(x=100,y=90)
    boutonint.place(x=100,y=40)

def integrale():
    fenetre1=Tk() #création d'une nouvelle fenêtre pour rentrer la valeur des bornes
    fenetre1.configure(bg = 'turquoise')
    fenetre1.geometry("300x150")
    point=StringVar()
    P=ttk.Entry(fenetre1,textvariable=point)
    P.place(x=10,y=50)
    tk.Label(fenetre1,text="Borne 1 ",fg='black', bg='turquoise', font=('arial',12)).place(x=50,y=10)
    pointimax=StringVar()

```

```

I=tkk.Entry(fenetre1,textvariable=pointimax)
I.place(x=160,y=50)
tk.Label(fenetre1,text="Borne 2",fg='black', bg='turquoise', font=('arial',12)).place(x=200,y=10)
def ok():
    try:
        Val1=P1.get()
        Val2=P.get()
        Val3=I.get()
        res=Polynome(eval(Val1)).integrale(eval(Val2),eval(Val3))
        resultat.set(str(res))
        afficher_latex(res)
        pas_de_graphique()
    except:
        afficher_latex("Erreur de saisie")
        fenetre1.destroy()
boutonok=Button(fenetre1, text="Calculer", width=20,command=ok)
boutonok.place(x=70,y=90)

```

#Création des boutons et de la fenêtre graphique

```

cadre=tk.Label(fenetre,borderwidth=4,relief="ridge", bg='beige',padx=415,pady=150)
boutonA= Button(fenetre, text = "Addition \n ",width=20 ,command=addition, bg = 'MediumOrchid1')
boutonS= Button(fenetre, text = "Soustraction \n ",width=20,command=soustraction, bg = 'MediumOrchid1')
boutonP = Button(fenetre, text = "Cacul pour une \n valeur de x",width=20 ,command=hornerPoint, bg = 'MediumPurple2')
boutonM = Button(fenetre, text = "Multiplication \n de 2 polynômes", width=20 ,command=multiplication, bg = 'MediumOrchid1')
boutonMx = Button(fenetre, text = "Multiplication \n par un réel", width=20 ,command=multiReel, bg = 'MediumPurple2')
boutonR2 = Button(fenetre, text = "Racines d'un polynôme \n de 2nd degré", width=20 ,command=racine2, bg = 'MediumPurple2')
boutonRev = Button(fenetre, text = "Racines évidentes \n ", width=20 ,command=evRacine, bg = 'MediumPurple2')
boutonD = Button(fenetre, text = "Division \n Quotient, Reste", width=20 ,command=division, bg = 'MediumOrchid1')
boutonIn = Button(fenetre, text = "Primitive \n ", width=20 ,command=primitiver, bg = 'MediumPurple2')
boutonDe = Button(fenetre, text = "Dérivée \n ", width=20 ,command=derivation, bg = 'MediumPurple2')
boutonSt = Button(fenetre, text = "Sturm \n ", width=20 ,command=Sturmm, bg = 'MediumPurple2')
boutonPg = Button(fenetre, text = "Pgcd \n ", width=20 ,command=pgcdd, bg = 'MediumOrchid1')
boutonDerSuc = Button(fenetre, text = "Dérivée successive \n ", width=20 ,command=DerSucc, bg = 'MediumPurple2')
boutonBezout = Button(fenetre, text = "Bézout \n Pol 1, Pol 2 ", width=20 ,command=bezoutt, bg = 'MediumOrchid1')
boutonPpcm = Button(fenetre, text = "Ppcm \n ", width=20 ,command=ppcmm, bg = 'MediumOrchid1')
boutonBairstow = Button(fenetre, text = "Bairstow \n ", width=20 ,command=bairstow, bg = 'MediumPurple2')
boutonIntégration = Button(fenetre, text = "Intégrale \n ", width=20 ,command=integrale, bg = 'MediumPurple2')

```

```

frame_graph = Frame(fenetre, width=350, height=250)

```

#Affichage des boutons et des espaces d'entrée

```

cadre.place(x=415,y=100)
P1.place(x=30,y=130)
P2.place(x=30,y=190)
label_resultat.place(x=500,y=450)

```

```

boutonP.place(x=480,y=120)
boutonA.place(x=480,y=170)
boutonM.place(x=480,y=220)
boutonR2.place(x=480,y=270)
boutonIn.place(x=480,y=320)
boutonPg.place(x = 480, y = 370)

```

```

boutonS.place(x=680,y=170)
boutonMx.place(x=680,y=220)
boutonRev.place(x=680,y=270)
boutonDe.place(x=680,y=320)
boutonPpcm.place(x = 680, y = 370)

```

```

boutonD.place(x=880,y=220)
boutonSt.place(x = 880, y = 270)
boutonDerSuc.place(x = 880, y = 320)
boutonBezout.place(x = 880, y = 370)

```

```

boutonBairstow.place(x = 1080, y = 270)
boutonIntégration.place(x = 1080, y = 320)

```

```

frame_graph.place(x=30, y=350)

```

#Zone de texte

```

tk.Label(text="Manipulation de Polynômes",fg='turquoise', bg='beige', font=('arial',20)).place(x=550,y=10)
tk.Label(text="Polynôme 1 sous la forme [[degrés,coefficient]]",bg='beige',font=('arial',12)).place(x=30,y=110)
tk.Label(text="Polynôme 2 sous la forme [[degrés,coefficient]] ",
justify="left",bg='beige',font=('arial',12)).place(x=30,y=160)
tk.Label(text="Résultat : ",fg='turquoise', bg='beige', font=('arial',15)).place(x=450,y=450)
tk.Label(text=" Opérations ",fg='turquoise', bg='beige', font=('arial',15)).place(x=450,y=85)
tk.Label(text="Graphique :",fg='turquoise',bg='beige',font=('arial',15)).place(x=30,y=325)

```

#Légende pour la couleur des boutons

```

tk.Label(fenetre, text="•", fg="MediumPurple2", bg="beige").place(x=1150, y=110)
tk.Label(fenetre, text="1 polynôme", bg="beige").place(x=1165, y=110)

```

```

tk.Label(fenetre, text="•", fg="MediumOrchid1", bg="beige").place(x=1150, y=130)

```

```

tk.Label(fenetre, text="2 polynômes", bg="beige").place(x=1165, y=130)

#Graphique
canvas_graph = None
def afficher_graphique(poly):
    global canvas_graph
    if canvas_graph is not None:
        canvas_graph.get_tk_widget().destroy()
    x = np.linspace(-10, 10, 400)
    y = [poly.point(val) for val in x]
    fig, ax = plt.subplots()
    ax.plot(x, y)
    ax.axhline(0, color='black', linewidth=1.5)
    ax.axvline(0, color='black', linewidth=1.5)
    fig.subplots_adjust(bottom=0.20)
    fig.subplots_adjust(left=0.15)
    canvas_graph = FigureCanvasTkAgg(fig, master=frame_graph)
    canvas_graph.draw()
    canvas_graph.get_tk_widget().place(x=0, y=0, width=350, height=250)

#Format LaTeX
def convertir_sympy(expr):
    x = symbols('x')
    if isinstance(expr, Polynome):
        return sum(c * x**d for d, c in expr.termes)
    return expr

from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

canvas_latex = None
def afficher_latex(expr):
    global canvas_latex
    if canvas_latex is not None:
        canvas_latex.get_tk_widget().destroy()
    fig, ax = plt.subplots()
    ax.axis("off")
    ax.text(0.5, 0.5, r"$" + latex(convertir_sympy(expr)) + r"$", fontsize=10, ha="center", va="center")
    #convertir_sympy() convertit le polynôme en expression SymPy
    #latex() transforme en code LaTeX
    #"$" indique que c'est du code LaTeX
    #r sert à construire une chaîne LaTeX correcte
    #les alignements verticaux et horizontaux sont centrés
    canvas_latex = FigureCanvasTkAgg(fig, master=fenetre)
    canvas_latex.draw()
    canvas_latex.get_tk_widget().place(x=545, y=450, width=660, height=25)

def afficher_latex_division(Q, R):
    global canvas_latex
    if canvas_latex is not None:
        canvas_latex.get_tk_widget().destroy()
    fig, ax = plt.subplots()
    ax.axis("off")
    x = symbols('x')
    exprQ = sum(c * x**d for d, c in Q.termes)
    exprR = sum(c * x**d for d, c in R.termes) if isinstance(R, Polynome) else R
    texte = r"${\text{}} " + latex(convertir_sympy(exprQ)) + ", " + r"\quad {\text{}} " + latex(convertir_sympy(exprR)) + r"$"
    ax.text(0.5, 0.5, texte, fontsize=10, ha="center", va="center")
    canvas_latex = FigureCanvasTkAgg(fig, master=fenetre)
    canvas_latex.draw()
    canvas_latex.get_tk_widget().place(x=545, y=450, width=660, height=25)

def afficher_latex_racinesEV(B, facteurs):
    global canvas_latex

```

```

if canvas_latex is not None:
    canvas_latex.get_tk_widget().destroy()

fig, ax = plt.subplots()
ax.axis("off")

x = symbols('x')

texte = B + "\n" + str(facteurs)

ax.text(0.5, 0.5, texte, fontsize=10, ha="center", va="center")

canvas_latex = FigureCanvasTkAgg(fig, master=fenetre)
canvas_latex.draw()
canvas_latex.get_tk_widget().place(x=545, y=450, width=660, height=50)

def pas_de_graphique():
    #affichage s'il n'y a pas de graphique à afficher
    global canvas_graph

    if canvas_graph is not None:
        canvas_graph.get_tk_widget().destroy()
        canvas_graph = None

    fig, ax = plt.subplots()
    ax.axis("off")
    ax.text(0.5, 0.5, "Pas de graphique disponible",
           ha="center", va="center", fontsize=10)

    canvas_graph_local = FigureCanvasTkAgg(fig, master=frame_graph)
    canvas_graph_local.draw()
    canvas_graph_local.get_tk_widget().place(x=0, y=0, width=350, height=250)

#Image pour les opérations
from PIL import Image, ImageTk

img = Image.open("px.png")      #charge l'image dans Python
img = img.resize((50, 40))      #définit la taille de l'image (largeur,hauteur)
photo = ImageTk.PhotoImage(img) #convertit l'image dans un format que tkinter sait afficher
label = tk.Label(fenetre, image=photo,bg='beige')
label.place(x=425,y=130)

img1 = Image.open("+ et -.png")
img1 = img1.resize((30, 40))
photo1 = ImageTk.PhotoImage(img1)
label1 = tk.Label(fenetre, image=photo1,bg='beige')
label1.place(x=430,y=170)

img2 = Image.open("x et div.png")
img2 = img2.resize((30, 40))
photo2 = ImageTk.PhotoImage(img2)
label2 = tk.Label(fenetre, image=photo2,bg='beige')
label2.place(x=430,y=220)

img3 = Image.open("racine.png")
img3 = img3.resize((30, 30))
photo3 = ImageTk.PhotoImage(img3)
label3 = tk.Label(fenetre, image=photo3,bg='beige')
label3.place(x=430,y=280)

img4 = Image.open("inté.png")
img4 = img4.resize((30, 30))
photo4 = ImageTk.PhotoImage(img4)
label4 = tk.Label(fenetre, image=photo4,bg='beige')
label4.place(x=420,y=330)

img5 = Image.open("d dx.png")
img5 = img5.resize((30, 30))
photo5 = ImageTk.PhotoImage(img5)
label5 = tk.Label(fenetre, image=photo5,bg='beige')
label5.place(x=445,y=330)

img6 = Image.open("engrenage.png")
img6 = img6.resize((30, 20))
photo6 = ImageTk.PhotoImage(img6)
label6 = tk.Label(fenetre, image=photo6,bg='beige')
label6.place(x=430,y=380)

```

